Lecture 08 — Problem Solving in Array & Vector

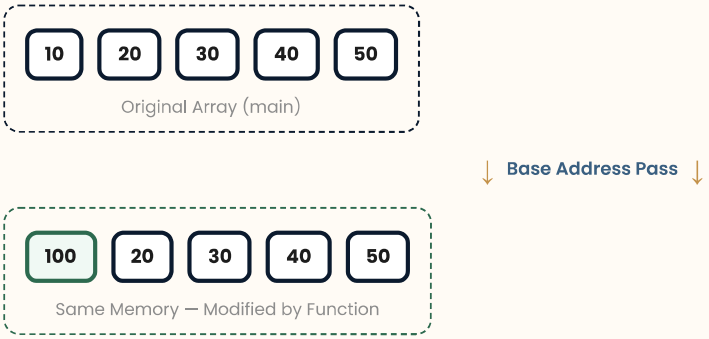
DSA Mastery Series | C++ Focused | Arrays, Strings & Two Pointers Track

Passing Array to Function

CORE INTUITION

C++ me jab array ko function me pass karte hain, to `array ka base address (pointer)` pass hota hai, na ki poora array copy hota hai. Isliye function ke andar kiye gaye changes **original array me reflect hote hain** — ye behavior **Pass by Reference** jaisa hota hai.

Memory View — Array Pass by Reference




```cpp
void printValue(int arr[], int n) {
    // Array print karo
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    // Original array modify hoga!
    arr[0] = 100;
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    printValue(arr, 5);   // Base address pass hota hai

    cout << arr[0] << endl;   // Output: 100 (modified!)
    return 0;
}
```

CRITICAL INSIGHT

`int arr[]` function parameter me likhne par bhi ye **pointer** hi hota hai (`int* arr` ke equivalent). `sizeof(arr)` function ke andar pointer ka size dega, array ka nahi. Isliye `size hamesha alag se pass karna padta hai` .

COMPLEXITY ANALYSIS

Metric	Value	Reason
Time	O(n)	Traversal for printing
Space	O(1)	No extra array created

◈ ◈ ◈

Problem 1 — LeetCode 268: Missing Number

PROBLEM STATEMENT

Given an array `nums` containing **n distinct numbers** in the range **[0, n]**. Return the only number that is missing from the array.

Example: nums = [3,0,1] → Range [0,3] me 0,1,2,3 hone chahiye → **2 missing hai**

Approach 1: Brute Force O(n²)

ALGORITHM

1. `i = 0` se `n` tak har expected number ko check karo.
2. Har `i` ke liye poore array me linear search karo.
3. Jo number array me nahi mile, wahi missing number hai.

```
int missingNumber(vector<int>& arr) {
    int n = arr.size();

    // 0 se n tak saare expected numbers check karo
    for (int i = 0; i <= n; i++) {
        bool found = false;

        // Current expected number ko poore array me search karo
        for (int j = 0; j < n; j++) {
            if (arr[j] == i) {
                found = true;
                break;
            }
        }

        // Number nahi mila → wahi missing number hai
        if (!found) return i;
    }
    return -1;
}
```

DRY RUN — BRUTE FORCE

nums = [3,0,1], n = 3

i	Search in Array	Found?	Action
0	[3,0,1]	Yes (index 1)	Continue
1	[3,0,1]	Yes (index 2)	Continue
2	[3,0,1]	No	**Return 2**

Approach 2: Sum Formula O(n)

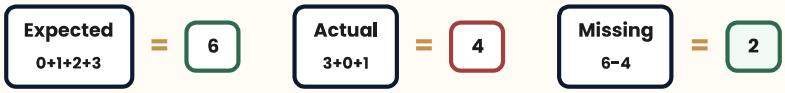
INTUITION / INSIGHT

0 se n tak numbers ka **Expected Sum** formula se nikal sakte hain: `n × (n+1) / 2`. Array ke elements ka **Actual Sum** nikalo. Dono ka difference hi missing number hoga.

```cpp
int missingNumber(vector<int>& arr) {
    int n = arr.size();
    int actualSum = 0;

    // Array ka Actual Sum calculate karo
    for (int i = 0; i < n; i++)
        actualSum += arr[i];

    // Expected Sum = 0+1+2+...+n
    int expectedSum = n * (n + 1) / 2;

    // Difference hi missing number hai
    return expectedSum - actualSum;
}
```

DRY RUN — SUM FORMULA

nums = [3,0,1], n = 3



WARNING

Agar numbers **bahut bade ho** ($n \approx 10^5$ ya zyada) to sum **integer overflow** kar sakta hai. C++ me `long long` use karo ya XOR approach prefer karo.

Approach 3: XOR (Optimal) O(n) — Best

INTUITION / INSIGHT

XOR ki magic property: `x ^ x = 0` aur `x ^ 0 = x`. Agar expected numbers (0 se n) ka XOR aur actual array numbers ka XOR kar dein, to same numbers ek dusre ko cancel kar denge. Jo bachega wahi missing number hoga.

```cpp
int missingNumber(vector<int>& arr) {
    int n = arr.size();
    int ans = n;   // n ko pehle include kar diya

    // Ek hi loop me Expected (i) aur Actual (arr[i]) ka XOR
    for (int i = 0; i < n; i++) {
        ans ^= i;       // Expected number
        ans ^= arr[i];  // Actual number
    }
    // Sirf missing number bachega
    return ans;
}
```

DRY RUN — XOR

nums = [3,0,1], n = 3, Initial ans = 3

i	ans ^= i	ans ^= arr[i]	ans
0	3 ^ 0 = 3	3 ^ 3 = **0**	0
1	0 ^ 1 = 1	1 ^ 0 = **1**	1
2	1 ^ 2 = 3	3 ^ 1 = **2**	2

✓ Final ans = 2 → Missing Number

COMPLEXITY COMPARISON

Approach	Time	Space	Pros	Cons
Brute Force	O(n²)	O(1)	Simple	Too slow
Sum Formula	O(n)	O(1)	Easy	Overflow risk
XOR (Optimal)	O(n)	O(1)	No overflow	Bitwise logic

◈ ◈ ◈

Problem 2 — GFG: Search an Element in Array

PROBLEM STATEMENT

Given an array `arr[]` and an integer `target`, return the **index** of target if it exists. Otherwise, return **-1**.

Example: arr = [1,2,3,4,5], target = 4 → **Index: 3**

ALGORITHM

1. Loop `0` se `n-1` tak chalao.
2. Har `arr[i]` ko `target` se compare karo.
3. Match milte hi `i` return karo.
4. Loop khatam ho jaye aur match na mile to `-1` return karo.

```
int search(int arr[], int n, int target) {
    // Array ke saare elements traverse karo
    for (int i = 0; i < n; i++) {
        // Target mil gaya
        if (arr[i] == target)
            return i;
    }
    // Target nahi mila
    return -1;
}
```

COMPLEXITY ANALYSIS

Case	Time	Explanation
Best	O(1)	Target pehle position pe
Worst	O(n)	Target last pe ya absent
Space	O(1)	No extra space

◈ ◈ ◈

Problem 3 — GFG: Reverse an Array

PROBLEM STATEMENT

Given an array `arr[]`, reverse the array **in-place** (extra array use nahi karna).

Example: [1,2,3,4,5] → **[5,4,3,2,1]**

INTUITION / INSIGHT

Opposite elements ko **swap** karte jao. Ek swap me 2 elements apni correct position par aa jate hain. Isliye sirf `n/2 swaps` hi required hote hain. Ye **Two Pointer** pattern ka classic example hai.

Two Pointer Swap Visualization


```
void reverseArray(int arr[], int n) {
    // Sirf half array tak swap karna hai
    for (int i = 0; i < n / 2; i++) {
        // Opposite elements swap karo
        int temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }
}
```

BEST PRACTICE / OPTIMIZATION

`i < n/2` condition se **double swap** avoid hota hai. Agar `i < n` rakhte to array wapas original ho jata (do baar reverse). Hamesha **half traversal** karo in-place reversal me.

COMPLEXITY ANALYSIS

Metric	Value	Reason
Time	O(n)	n/2 iterations
Space	O(1)	Only temp variable, in-place

Problem 4 — GFG: Check if Array is Sorted

PROBLEM STATEMENT

Given an array `arr[]`, check whether it is sorted in **non-decreasing order**. Return `true` if sorted, otherwise `false`.

Example 1: [1,2,3,4,5] → **true** | **Example 2:** [5,4,3,2,1] → **false**

ALGORITHM

1. Loop 1 se `n-1` tak chalao.
2. Har `arr[i]` ko `arr[i-1]` se compare karo.
3. Agar `arr[i] < arr[i-1]` → order break ho gaya → `false`.
4. Loop complete ho jaye → `true`.

```
bool arraySortedOrNot(int arr[], int n) {
    // First element ka koi previous nahi hota
    for (int i = 1; i < n; i++) {
        // Order break ho gaya
        if (arr[i] < arr[i - 1])
            return false;
    }
    // Array sorted hai
    return true;
}
```

EDGE CASE / PITFALL

- **Single element array** → hamesha sorted hoti hai (loop execute nahi hoga, directly `true`).
- **Equal elements** → allowed hain (< check hai, <= nahi). [1,2,2,3] → sorted.
- **Empty array** → technically sorted (`true`).

COMPLEXITY ANALYSIS

Case	Time	Explanation
Best	O(1)	Pehle comparison pe hi break
Worst	O(n)	Array sorted hai, pura check karna padta hai
Space	O(1)	No extra space

Vector in C++ — Dynamic Array

CORE INTUITION

Vector C++ ka **dynamic array** hai. Static array ki tarah fixed size nahi hota — runtime pe **grow** aur **shrink** ho sakta hai. Ye `STL (Standard Template Library)` ka part hai aur arrays se zyada flexible aur safe hai.

Declaration Methods

```cpp
// Method 1: Empty Vector
vector<int> arr;                    // [ ]

// Method 2: Fixed Size (default value = 0)
vector<int> arr1(5);                // [0, 0, 0, 0, 0]

// Method 3: Fixed Size with Custom Value
vector<int> arr2(5, 100);           // [100, 100, 100, 100, 100]

// Method 4: Direct Initialization
vector<int> arr3 = {10,20,30,40,50};  // [10, 20, 30, 40, 50]

// Method 5: Copy Constructor
vector<int> arr4(arr3);             // [10, 20, 30, 40, 50]
```

Common Operations

OPERATIONS CHEATCODE

Operation	Syntax	Time	Description
Insert at end	`arr.push_back(x)`	Amortized O(1)	Last me element add karta hai
Remove last	`arr.pop_back()`	O(1)	Last element remove karta hai
Size	`arr.size()`	O(1)	Total elements return karta hai
First element	`arr.front()`	O(1)	Pehla element
Last element	`arr.back()`	O(1)	Aakhri element
Check empty	`arr.empty()`	O(1)	true agar vector empty hai
Access	`arr[i]` / `arr.at(i)`	O(1)	Index se access (at() safer)

```cpp
int main() {
    vector<int> arr;

    // Insert elements (hamesha last me)
    arr.push_back(10);   // [10]
    arr.push_back(20);   // [10, 20]
    arr.push_back(30);   // [10, 20, 30]
    arr.push_back(40);   // [10, 20, 30, 40]

    // Remove last element
    arr.pop_back();      // [10, 20, 30]

    // Size
    cout << "Size: " << arr.size() << endl;   // 3

    // First & Last
    cout << "First: " << arr.front() << endl;  // 10
    cout << "Last: " << arr.back() << endl;    // 30

    // Check empty
    if (arr.empty())
        cout << "Empty";
    else
        cout << "Not Empty";   // Not Empty

    return 0;
}
```

CRITICAL INSIGHT

`push_back()` agar **capacity** full ho jaye to vector **double capacity** allocate karke sab elements copy karta hai. Ye **amortized O(1)** hota hai — average case me fast, par kabhi-kabhi O(n) lag sakta hai jab resize ho. Isliye `arr.reserve(n)` use karo agar size pehle se pata ho.

INTERVIEW TIP

Interviewer pooch sakta hai: *"Array vs Vector me difference kya hai?"*

Answer: Array **fixed size** hota hai (stack memory), Vector **dynamic size** hota hai (heap memory). Vector **automatic resizing**, **bounds checking** (at()), aur **STL algorithms** ke saath compatible hota hai.

Pattern Transfer / Related Problems

PATTERN TRANSFER

Problem	Pattern Used	Complexity
Missing Number (XOR)	Bit Manipulation	O(n) / O(1)
Find Duplicate Number	Floyd's Cycle / XOR / Sum	O(n) / O(1)
Reverse Array	Two Pointer Swap	O(n) / O(1)
Rotate Array by K	Reverse 3 times	O(n) / O(1)
Check Sorted	Linear Comparison	O(n) / O(1)
First & Last Occurrence	Linear Search / Binary Search	O(n) / O(log n)
Vector: Merge 2 Sorted Arrays	Two Pointer	O(n+m) / O(n+m)

Key Takeaways

TAKEAWAY / SUMMARY

- Array function me **base address** pass hota hai → original array modify ho sakta hai.
- Missing Number: **Sum Formula** (O(n)) easy hai par overflow risk hai. **XOR** (O(n)) optimal aur safe hai.
- Linear Search: **Unsorted data** ke liye O(n) — pehla match return karo, `break` se optimize karo.
- Reverse Array: **Two Pointer Swap**, sirf n/2 iterations, in-place O(1) space.
- Check Sorted: `arr[i] < arr[i-1]` check karo, pehle break pe `false` return.
- Vector = **Dynamic Array**. `push_back()`, `pop_back()`, `size()`, `front()`, `back()` — sab O(1).
- Vector **amortized O(1)** insert — capacity double hoti hai jab full ho.

◆ QUICK REVISION CHEATSHEET ◆

Array to Function
Base address pass hota hai (pointer).
Changes original array me reflect hote hain.
Size alag se pass karna padta hai.

Missing Number — XOR (Optimal)
```
ans = n;
for(i=0; i<n; i++) ans ^= i ^ arr[i];
```
TC: O(n) SC: O(1) No Overflow

Missing Number — Sum
```
expected = n*(n+1)/2;
actual = sum(arr);
return expected - actual;
```
⚠ Overflow risk for large n

Linear Search
```
for(i=0; i<n; i++) if(arr[i]==target) return i;
return -1;
```
TC: O(n) worst SC: O(1)

Reverse Array (Two Pointer)
```
for(i=0; i<n/2; i++)
swap(arr[i], arr[n-1-i]);
```
TC: O(n) SC: O(1) in-place

Check Sorted
```
for(i=1; i<n; i++)
if(arr[i] < arr[i-1]) return false;
return true;
```
TC: O(n) SC: O(1)

Vector Operations
`push_back(x)` → Last insert (Amortized O(1))
`pop_back()` → Last remove (O(1))
`size(), front(), back(), empty()` → O(1)
`vector<int> v(n, val)` → Init with size n

Common Mistakes
- Array function me pass karke size nahi bhejna
- Reverse me `i < n` rakho to double reverse hoga
- Sum formula me integer overflow
- Vector ka `capacity != size` — confuse mat karo

Interview Questions
- Array vs Vector difference?
- Missing Number me XOR ka proof?
- Reverse array recursive kaise karein?
- Vector ka internal working (resize/capacity)?

XOR Properties (Must Remember)
x ^ x = 0 | x ^ 0 = x
x ^ y = y ^ x (Commutative)
(x ^ y) ^ z = x ^ (y ^ z) (Associative)
Missing Number, Duplicate, Single Number — sab me kaam aata hai